

Proyecto ICAP



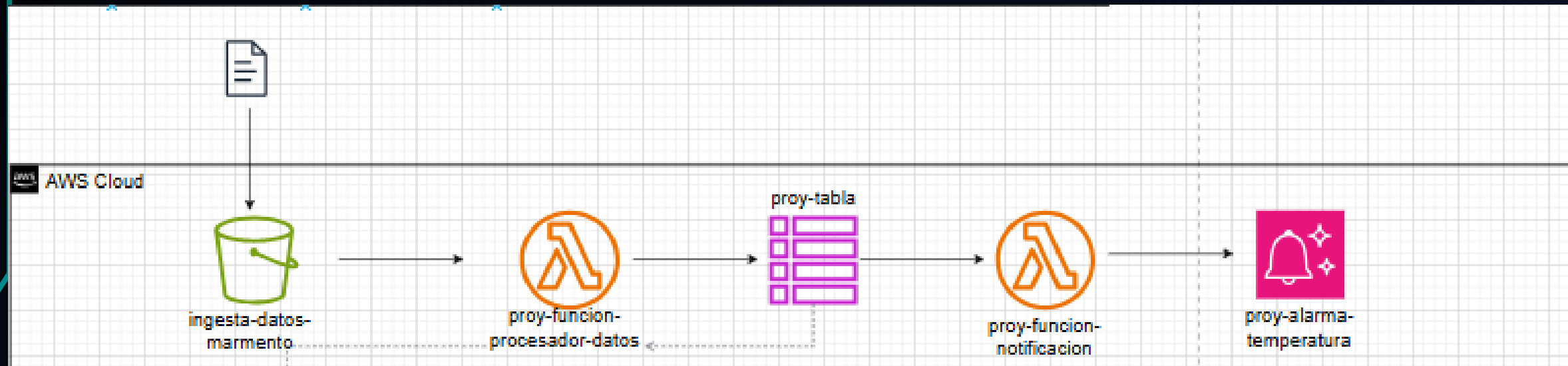
REALIZADO POR:

-RUBÉN DEL VALLE CARRILLO
-ÁNGEL GONZALÉZ DE SANTOS

ÍNDICE

- Creación del pipeline
- Creación del servicio web
- Implementación de métodos de escalabilidad
- Conclusión

CREACIÓN DEL PIPELINE



CREACION DEL PIPELINE

- proy-funcion-procesador-datos
- proy-funcion-notificacion

```
for row in csv_reader:
    # Formato esperado del CSV: Fecha, Medias, Desviaciones
    fecha = row['Fecha'] # Ej: 22/03/2017
    media = row['Medias']
    desviacion = row['Desviaciones']

    partes_fecha = fecha.split('/')
    if len(partes_fecha) == 3:
        dia_normalizado = int(partes_fecha[2])
        mes_normalizado = int(partes_fecha[1])
        anio = partes_fecha[0]
        mes_ano = f"{mes_normalizado}-{anio}" # Ej: "03-2017" se convierte en "3-2017"
        fecha = f"{anio}/{mes_normalizado}/{dia_normalizado}" # Ej: "2017-03-22" se convierte en "2017-3-22"
    else:
        print(f"Formato de fecha incorrecto: {fecha}")
        continue
```

```
if desviacion_val > 0.5:
    mensaje = (
        f"ALERTA AQUASENSE:\n"
        f"Fecha: {fecha}\n"
        f"Desviación detectada: {desviacion_val}\n"
        f"Umbral permitido: 0.5"
    )
    sns = boto3.client('sns')

    snsTopicArn = 'arn:aws:sns:us-east-1:949076425070:proy-alarma-temperatura'

    sns.publish(
        TopicArn=snsTopicArn,
        Message=mensaje,
        Subject='¡Alerta Crítica Mar Menor!'
    )
    print(f"Alerta enviada para fecha {fecha}")
    alertas_enviadas += 1
```

CREACION DEL SERVICIO WEB

- Creación de infraestructura básica

-VPC

-Dos subredes publicas

-Tres grupos de seguridad

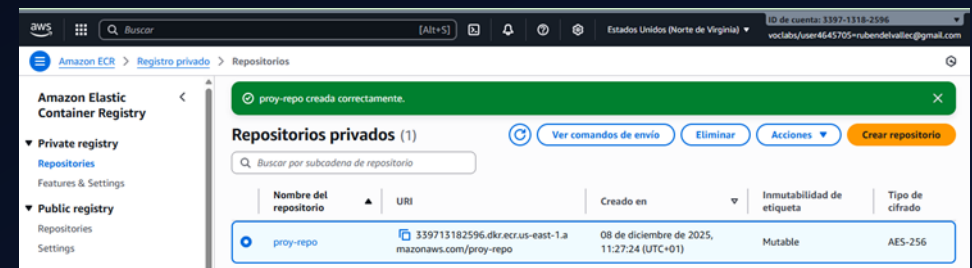
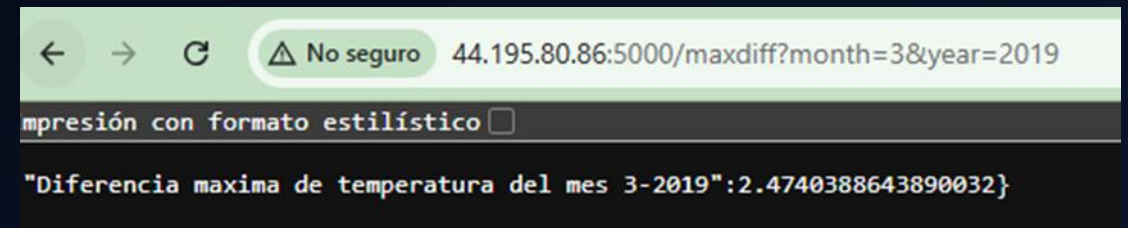
-Instancia Docker

```
140  ## Instancia EC2 de pruebas
141  DockerInstance:
142    Type: AWS::EC2::Instance
143    Properties:
144      InstanceType: t2.micro
145      ImageId: ami-08a0d1e16fc3f61ea
146      KeyName: proy-keypair
147      IamInstanceProfile: LabInstanceProfile
148      SubnetId: !Ref PublicSubnet1
149      SecurityGroupIds:
150        - !Ref FlaskSecurityGroup
151      Tags:
152        - Key: Name
153          Value: DockerInstance
154      UserData:
155        Fn::Base64: !Sub |
156          #!/bin/bash
157          dnf update -y
158          dnf install -y docker
159          systemctl enable docker
160          systemctl start docker
161  DiskVolume:
162    Type: AWS::EC2::Volume
163    Properties:
164      Size: 100
165      AvailabilityZone: !GetAtt DockerInstance.AvailabilityZone
166      DeletionPolicy: Snapshot
167
168  DiskMountPoint:
169    Type: AWS::EC2::VolumeAttachment
170    Properties:
171      InstanceId: !Ref DockerInstance
172      VolumeId: !Ref DiskVolume
173      Device: /dev/sdh
174
```

CREACION DEL SERVICIO WEB

- Implementación de función Flask.
- Creación de nuevo directorio
- Creación de los ficheros
- Creación de imagen
- Creación de repositorio ECR

```
[ec2-user@ip-192-168-1-33 flask_docker]$ sudo docker build -t flask_container .  
[+] Building 0.2s (10/10) FINISHED  
=> [internal] load build definition from Dockerfile  
=> => transferring dockerfile: 641B  
=> [internal] load metadata for docker.io/library/python:3.8-alpine  
=> [internal] load .dockerignore  
=> => transferring context: 2B  
=> [1/5] FROM docker.io/library/python:3.8-alpine@sha256:3d93b1f77efce339aa77db726656872517b0d67837989aa7c4b35bd5ae7e81ba  
=> [internal] load build context  
=> => transferring context: 368B  
=> CACHED [2/5] WORKDIR /app  
=> CACHED [3/5] COPY requirements.txt requirements.txt  
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt  
=> CACHED [5/5] COPY . .  
=> exporting to image  
=> => exporting layers  
=> => writing image sha256:64d6e714ac76fd323c2a3d20e8ab73e7485f2a456d0b68596dff5a8f640630db  
=> => naming to docker.io/library/flask_container  
[ec2-user@ip-192-168-1-33 flask_docker]$ sudo docker run -p 5000:5000 -d flask_container  
d2192de50a78b358efe7655af54771dc7b77af100cldf799960620eb061aa614
```



CREACION DEL SERVICIO WEB

```
mes_ano = f"{m}-{y}"
response = table.scan(
    FilterExpression=Attr("mes_ano").eq(mes_ano)
)
items = response.get('Items', [])
if not items:
    return jsonify({"error": f"Datos no encontrados para {mes_ano}"}, 404)
```

-/maxdiff

```
for i in items:
    if float(i['media']) > max_act:
        max_act = float(i['media'])

if m == 1:
    m_2 = 12
    y_2 = y-1
else:
    m_2 = m-1
    y_2 = y

mes_ano_ant = f"{m_2}-{y_2}"
response_ant = table.scan(
    FilterExpression=Attr("mes_ano").eq(mes_ano_ant)
)
items_ant = response_ant.get('Items', [])
if not items_ant:
    return jsonify({"error": f"Datos no encontrados para mes anterior"}, 404)

max_ant = 0
for i in items_ant:
    if float(i['media']) > max_ant:
        max_ant = float(i['media'])

max_diff = max_act-max_ant
return jsonify({ f"Diferencia maxima de temperatura del mes {mes_ano}": max_diff
```

-/sd

```
max = 0
for i in items:
    if float(i['desviacion']) > max:
        max = float(i['desviacion'])

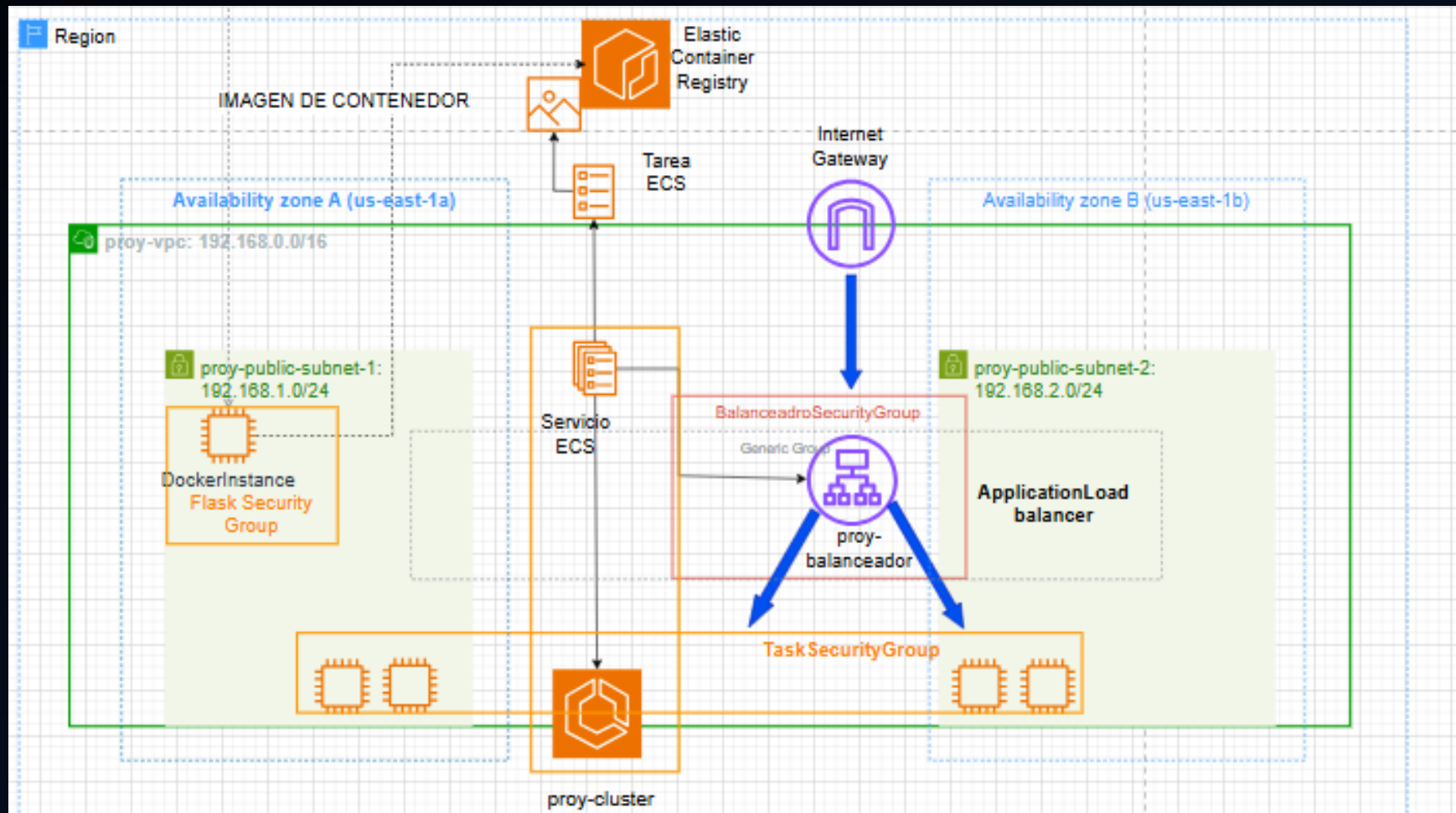
return jsonify({
    f"Maxima desviacion del mes {mes_ano}": max
})
```

-/temp

```
def media(items, field: str):
    """
    Calcula la media de una lista numérica almacenada en el atributo 'field'
    (lista de Decimal / float / int).
    """
    # Convertimos cada valor a float por si vienen como Decimal
    valores = []
    for i in items:
        valores.append(float(i[field]))

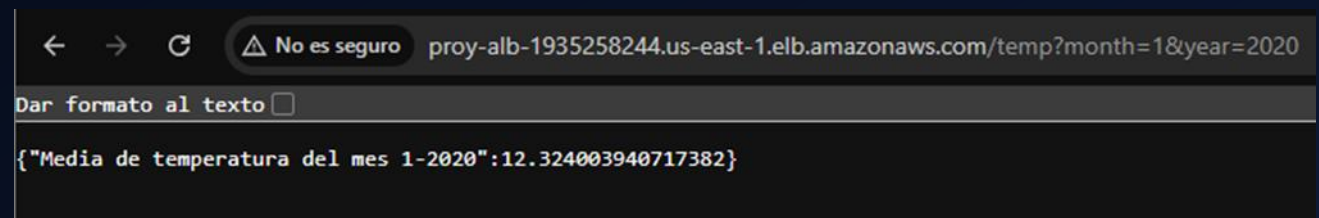
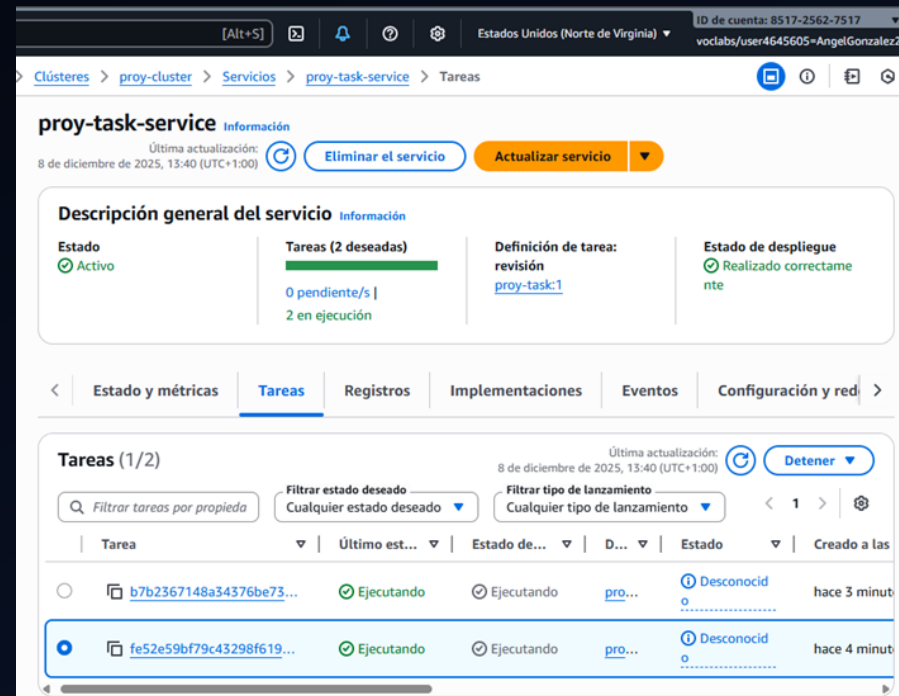
    media_global = sum(valores) / len(valores)
    return media_global
```

IMPLEMENTACIÓN DE MÉTODOS DE ESCALABILIDAD

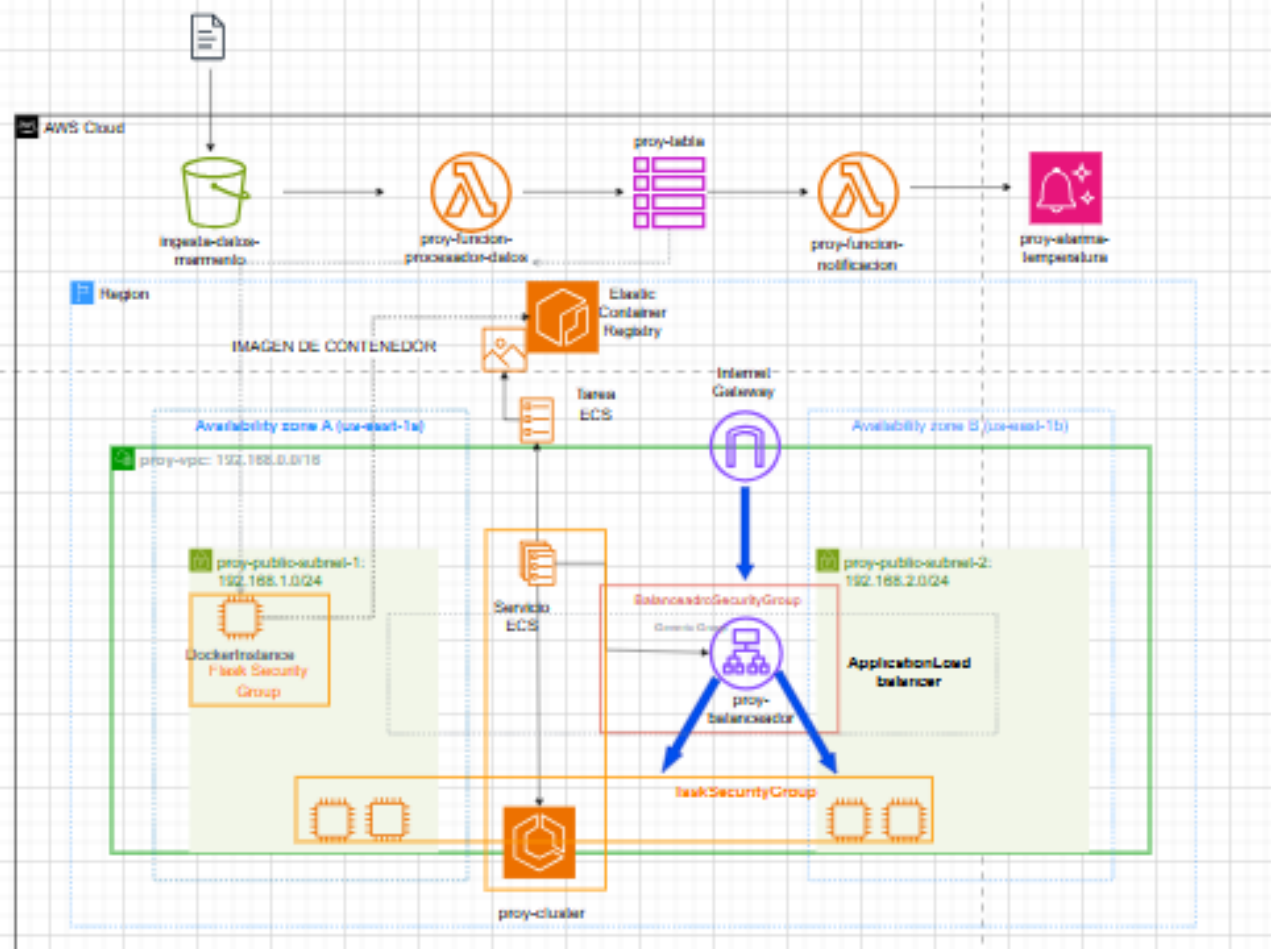


IMPLEMENTACIÓN DE MÉTODOS DE ESCALABILIDAD

- Creación de puerta de enlace
- Creación de balanceador
- Creación de Clúster
- Definición de tareas
- Creación del Servicio



Ángel González de Santos, Rubén Del Valle Carrillo



CONCLUSIONES

- CLAVE DE PARTICIÓN
- FUNCIONES
- PUERTO DE ENLACE